OmicsLake: Versioned Data Management with Automatic Lineage Tracking for Reproducible Bioinformatics

Yusuke Matsui[1,2,*]

[1]Institute for Glyco-core Research (iGCORE), Nagoya University, Nagoya, Japan
[2]Biomedical and Health Informatics Unit, Graduate School of Medicine, Nagoya University, Nagoya, Japan
[*]Corresponding author: matsui.yusuke.d4@f.mail.nagoya-u.ac.jp

Abstract

Background: Reproducibility in bioinformatics is often compromised by weak traceability of intermediate data states. Typical RNA-seq and multi-omics workflows iteratively transform data through normalization, filtering, modeling, and enrichment analysis, but these transitions are usually tracked manually. Code version control addresses scripts, not evolving analytical datasets. As a result, reconstructing the exact data state used for a specific result remains difficult in routine exploratory analysis.
Findings: OmicsLake is an R package for snapshot-oriented data-state management with automatic dataset-level lineage capture in dplyr workflows. The implementation combines DuckDB (primary local persistence and query execution), Apache Arrow (in-memory interchange), and Parquet (export/interchange format). Core operations are exposed through a compact API (`put`, `get`, `snap`, `restore`, `tree`) designed for low-friction integration into existing analyses. In benchmark settings aligned with submitted figures, OmicsLake showed faster import, aggregation, and join operations than representative baseline R workflows, while snapshot creation was slower than a file-copy proxy due to explicit in-database backup materialization.
Conclusions: OmicsLake achieved 100% observed restoration success in RT-001 (30/30 iterations; 95% CI: 0.88–1.00) using strict structural and numerical equivalence checks. Simulated cross-environment transfer tests (RT-003; 3 logical pairs, 3 iterations each) also achieved complete equivalence within predefined tolerances. These results indicate that OmicsLake can strengthen data-state reproducibility in interactive bioinformatics workflows while remaining compatible with Git-, renv-, and container-based practices.
Software URL: https://github.com/matsui-lab/OmicsLake

Keywords: reproducibility, data lineage, version control, bioinformatics, R package, DuckDB, Parquet

1 Introduction

Reproducibility is a cornerstone of scientific research, yet computational biology faces persistent challenges in achieving reproducible analyses [1, 2]. Modern omics studies generate increasingly complex data: a single-cell RNA sequencing experiment may produce expression profiles for tens of thousands of cells, while multi-omics studies integrate transcriptomic, proteomic, and epigenomic layers. A typical RNA-seq analysis pipeline involves multiple interdependent steps—quality control, read alignment, quantification, normalization, batch correction, differential expression analysis, and pathway enrichment—each producing intermediate datasets that may be filtered, transformed, or subset based on exploratory findings [3]. The Bioconductor ecosystem [4, 5] provides a comprehensive framework for omics data analysis with standardized containers such

as SummarizedExperiment [6], yet no integrated, transparent mechanism exists for versioning intermediate analysis states within standard Bioconductor workflows. When a collaborator asks "which normalized counts did you use for this figure panel?" or when a reviewer requests re-analysis with modified filtering thresholds, reconstructing the exact data state becomes a time-consuming archaeological exercise. Without systematic tracking, reproducing the precise analytical state at any point in a project's history is practically impossible.

To clarify OmicsLake's position in the ecosystem, we categorize existing tools into four problem domains (Table 1):

(A) Workflow Provenance Systems Workflow managers such as **Snakemake** [7] and **Nextflow** [8] provide robust provenance tracking through declarative pipeline specifications. These tools excel at orchestrating complex, multi-step pipelines with automatic dependency resolution and parallelization. However, they require upfront workflow definition in domain-specific languages (Snakemake's Python-based rules or Nextflow's Groovy DSL), fundamentally conflicting with exploratory bioinformatics analysis where hypotheses evolve iteratively. Within the R ecosystem, **targets** [9] and its predecessor **drake** [10] provide similar workflow orchestration with native R syntax and automatic dependency detection through static code analysis. While targets elegantly handles function-level provenance, it operates at the pipeline execution level rather than the data state level—users cannot easily restore intermediate data states without re-executing the pipeline.

(B) Artifact Caching Several tools focus on caching computational artifacts to avoid redundant recomputation. Both **targets** and **drake** include sophisticated caching mechanisms that store function outputs keyed by input hashes and code signatures. The **memoise** package [11] provides function-level memoization for expensive computations. However, these caching systems are tightly coupled to specific execution contexts and do not provide independent, portable data versioning—cached objects cannot easily be extracted, shared, or queried outside the workflow system that created them.

(C) Data Catalogs and Storage Backends Tools like **pins** [12] and **arrow::open_dataset()** provide mechanisms for organizing and accessing datasets across local and remote storage. Pins offers a simple "pin and retrieve" interface with versioning support, but lacks automatic lineage tracking and in-process query capabilities. Arrow datasets enable efficient access to partitioned Parquet files but do not provide versioning or provenance features. **Git LFS** (Large File Storage) [13] extends Git to handle large binary files by storing file contents on a remote server while tracking lightweight pointers in the repository. However, Git LFS requires explicit file registration, lacks native support for tabular data queries, and imposes bandwidth costs for each version retrieval. **DVC** (Data Version Control) [14] provides Git-like semantics for data files with support for remote storage backends (S3, GCS, Azure). While DVC excels at pipeline reproducibility through its `dvc.yaml` specification, it operates at the file level rather than the data structure level, requiring serialization/deserialization overhead for each access. DVC's Python-centric design also creates friction for R users who must bridge language boundaries or use wrapper packages.

(D) Data Versioning with Lineage (OmicsLake) OmicsLake occupies a distinct niche: *interactive data versioning with automatic lineage tracking.* Unlike workflow managers (A), OmicsLake does not require pipeline predefinition—it captures data states during exploratory sessions. Unlike artifact caches (B), versioned data is independent and queryable via SQL. Unlike data catalogs (C), OmicsLake automatically propagates lineage metadata through dplyr operations. The **renv** package [15] provides complementary functionality for R package version management but does not address data versioning.

Table 1: Problem space categorization: existing tools address different aspects of reproducibility

Category	**Primary Focus**	**Representative Tools**	**Limitation for Interactive Analysis**
(A) Workflow Provenance	Pipeline execution order, dependency resolution	Snakemake, Nextflow, targets, drake	Requires predefined pipeline; no ad-hoc exploration
(B) Artifact Caching	Avoid recomputation of unchanged targets	targets cache, drake cache, memoise	Coupled to execution context; not portable
(C) Data Catalogs	Organize and access datasets	pins, Arrow datasets, Git LFS, DVC	No automatic lineage; file-level granularity
(D) Data Versioning	Interactive state capture with lineage	**OmicsLake**	—

Table 2 further contrasts representative toolchains across interactive-session support, lineage automation model, and deployment characteristics to make scope boundaries explicit.

Table 2: Feature comparison of data versioning approaches for bioinformatics workflows

Feature	**Git LFS**	**DVC**	**Snakemake**	**Nextflow**	**targets**	**pins**	**OmicsLake**
R native integration	No	Partial	Partial	No	Yes	Yes	Yes
Interactive session support	No	No	No	No	Partial	Yes	Yes
Automatic lineage tracking	No	No	Yes*	Yes*	Yes*	No	Yes
In-process query engine	No	No	No	No	No	No	Yes
Zero-setup deployment	No	No	No	No	Yes	Yes	Yes
Columnar compression	No	No	No	No	No	Partial	Yes
Portable versioned data	No	Yes	No	No	No	Yes	Yes

*Requires predefined workflow/target specification. "Automatic lineage" denotes propagation without manual edge annotation during routine operations under each tool's native abstraction (rule graph for workflow systems, dataset graph for OmicsLake).

The reproducibility crisis in computational science has been extensively documented [16, 17], yet existing tools address only portions of the problem. The ACM defines three distinct levels of reproducibility [18]: *repeatability* (same team, same experimental setup), *reproducibility* (different team, same experimental setup), and *replicability* (different team, different experimental setup). Current solutions typically focus on code reproducibility through version control or environment reproducibility through containerization, but leave data state reproducibility—the ability to restore exact intermediate analysis states—largely unaddressed.

We present OmicsLake, an R package designed to fill this critical gap by providing:

- **Deterministic state restoration**: Snapshot labels capture complete project states and enable reproducible rollback of table/object references
- **Automatic dependency tracking**: Dataset-level lineage graphs are constructed transparently through dplyr integration, achieving 100% observed edge recall in our validation experiments
- **Simulated transfer portability**: Labeled snapshots can be transferred and verified across logical environment-pair boundaries (RT-003)
- **FAIR-aligned provenance**: Internal metadata tables preserve findability, accessibility, interoperability, and reusability attributes for local analysis artifacts

2 Implementation

2.1 Architecture

OmicsLake is implemented as an R6 class-based package providing a unified interface to three complementary backend technologies (Figure 1).

Storage and Exchange Layer: DuckDB + Parquet DuckDB serves as the primary local persistence and query engine, providing an embedded OLAP (Online Analytical Processing) database that runs entirely within the R process without external server dependencies [19]. This architecture eliminates network latency and simplifies deployment—users install a single R package with no additional infrastructure requirements. DuckDB's vectorized execution engine processes data in batches of 1024–2048 tuples, enabling efficient CPU cache utilization and SIMD (Single Instruction, Multiple Data) parallelism for analytical workloads.

For data sharing and external interoperability, OmicsLake supports Parquet export with configurable compression (Snappy, Zstd, LZ4, Brotli). The `ol_export_parquet()` function provides fine-grained control:

Listing 1: Parquet export with compression options

```
# Export with Zstd compression (better ratio, slightly slower)
lake$export("expression_matrix", "data/expr.parquet",
            compression = "zstd", compression_level = 3)

# Row group size affects memory usage during read
ol_export_parquet("counts", "counts.parquet",
                  row_group_size = 50000)  # 50K rows per group
```

For exported artifacts, Parquet's columnar layout provides practical read-efficiency benefits: when selecting k columns from a table with m total columns, the bytes read are approximately proportional to k/m of total table bytes (assuming comparable per-column widths), whereas row-oriented formats such as CSV or RDS typically require scanning all columns.

Data Transfer: Apache Arrow Arrow provides an efficient data interchange layer between R and DuckDB [20]. When writing data via `lake$put()`, the implementation first converts the R data frame to an Arrow Table, then registers this table with DuckDB:

Listing 2: Internal data flow for put() operation (active table write path)

```
# Simplified internal implementation
.put_table <- function(name, data) {
  tbl <- arrow::arrow_table(data)  # R -> Arrow (memory copy)
  tmp <- paste0("ol_tmp_", timestamp())
  duckdb::duckdb_register_arrow(conn, tmp, tbl)  # Zero-copy
  DBI::dbExecute(conn, sprintf(
    "CREATE OR REPLACE TABLE %s AS SELECT * FROM %s",
    ident, tmp))
  duckdb::duckdb_unregister_arrow(conn, tmp)
}
```

The conversion from R data frame to Arrow Table incurs a single memory copy, but subsequent transfers between Arrow and DuckDB are zero-copy operations sharing the same memory buffers. This design is particularly beneficial for large datasets where minimizing memory copies is critical. Importantly, this listing shows the *active table* write path only. Restorable states are created by `snap()` through labeled backup tables and reference metadata (see "Data Model and Storage Semantics"), rather than by per-`put()` table-version identifiers.

Design Choice: R6 Classes We chose R6 over S4 or Reference Classes for several architectural reasons:

1. **Reference semantics**: The Lake object maintains mutable state (database connections, cached metadata) that persists across method calls. R6's reference semantics match users' mental model of a persistent data store without requiring explicit reassignment (`lake <- lake$put(...)`).

2. **Resource management**: The `finalize()` method provides automatic cleanup when the Lake object is garbage-collected, ensuring database connections are properly closed:

Listing 3: Automatic resource cleanup via finalize()

```
finalize = function() {
  if (!is.null(private$.state$conn)) {
    if (DBI::dbIsValid(private$.state$conn)) {
      DBI::dbDisconnect(private$.state$conn, shutdown = FALSE)
    }
  }
}
```

3. **Encapsulation**: Private fields (prefixed with `.`) hide implementation details while public methods form a stable API contract. This separation enables internal refactoring without breaking user code.

4. **Familiar syntax**: The `lake$method()` syntax reduces friction for users familiar with Python, JavaScript, or other object-oriented languages.

Terminology To ensure clarity, we define the following core concepts:

Version A storable data state identified by backend-specific references. For table data, OmicsLake uses labeled snapshot copies. For object data, OmicsLake stores append-only rows keyed by `version_ts`.

Snapshot A labeled checkpoint capturing the complete state of all datasets in a lake at a specific moment. Created via `snap()`, snapshots enable reproducible restoration of project states via `restore()`.

Lineage The directed acyclic graph (DAG) of dependencies between *dataset names* (table/object/view nodes), recording derivation relationships through tracked operations. Visualized via `tree()`.

In short: table states are represented by snapshot labels mapped to backup tables, whereas object states are represented by `version_ts`-indexed rows in `__ol_objects`.

2.2 Core API

The OmicsLake API follows a minimalist design philosophy: a small set of orthogonal operations that compose naturally. Table 3 summarizes the primary methods with their computational complexity.

Table 3: OmicsLake Core API Functions. Complexity estimates assume typical workloads with sequential table scans; actual performance depends on data characteristics, available indexes, and system resources.

Method	Description	Typical Complexity
`Lake$new(project)`	Initialize a new lake with DuckDB backend	$O(1)$
`lake$put(name, data)`	Store data with automatic type detection and lineage recording	$O(n)$[a]
`lake$get(name, ref)`	Retrieve data, optionally at a snapshot label or relative reference	$O(n)$[a]
`lake$ref(name)`	Get lazy reference for deferred query execution	$O(1)$
`save_as(.data, name, lake)`	Persist dplyr pipeline outputs while carrying inferred dependencies	$O(n)$[a]
`ol_commit(note, params)`	Store analysis notes/parameters for audit trail and repair context	$O(1)$
`lake$snap(label)`	Create named snapshot of current state (table backup + object label mapping)	$O\left(\sum_{i=1}^{k} n_i\right)$[b]
`lake$tree(name)`	Display lineage DAG via BFS traversal	$O(V+E)$[c]
`lake$restore(label)`	Restore project to labeled snapshot	$O\left(\sum_{i=1}^{k} n_i\right)$[b]

[a] n = rows in table; [b] k = number of captured tables, n_i = rows in table i; [c] V = dataset nodes, E = dependency edges

Type Detection and Storage Routing The `put()` method automatically routes data to appropriate storage backends based on type detection:

Listing 4: Automatic type detection in put()

```
.detect_storage_type = function(data) {
  if (inherits(data, "SummarizedExperiment")) return("se")
  if (inherits(data, "Seurat")) return("seurat")
  if (is.data.frame(data) || is.matrix(data)) return("table")
  return("object")  # Fallback: serialize as RDS blob
}
```

Tabular data is stored natively in DuckDB, enabling SQL queries and efficient columnar access. Arbitrary R objects (lists, S4 objects, functions) are serialized using R's native `serialize()` and stored as BLOBs in a metadata table. This dual-storage approach balances query performance for structured data with flexibility for complex objects.

Lazy Evaluation via ref() The `ref()` method returns a `tbl_lazy` object that defers execution until explicitly collected. This enables DuckDB's query optimizer to push filters and projections down to the storage layer:

Listing 5: Lazy evaluation enables query pushdown

```
# Query is not executed until collect()
result <- lake$ref("expression") |>
  dplyr::filter(gene_type == "protein_coding") |>
  dplyr::select(gene_id, sample_1, sample_2) |>
  dplyr::mutate(ratio = sample_1 / sample_2) |>
  dplyr::filter(ratio > 2) |>
  dplyr::collect()

# DuckDB optimizes: only reads required columns,
```

```
# applies both filters in single scan
```

Reference Syntax OmicsLake supports multiple reference formats for accessing historical data states, organized into two categories:

Snapshot Label: User-defined name created via `snap()` (e.g., `"v1.0_complete"`, `"submission_final"`). Snapshot labels capture project state and are resolved through `__ol_refs`.

Relative Reference: Built-in shortcuts for common access patterns:

- `"@latest"` — most recent version (default when ref omitted)
- `"@first"` — earliest stored object version (object reads)

The following examples demonstrate the unified syntax:

Listing 6: Version reference syntax examples

```
lake$get("counts")                  # Latest version (default)
lake$get("counts", "@latest")       # Explicit latest
lake$get("counts", "v1.0_complete") # Snapshot label
lake$get("model_obj", "@first")     # First stored object version
```

Note: Legacy `@tag(name)` syntax (e.g., `"@tag(raw)"`) is retained for compatibility but direct snapshot labels are recommended in new code.

2.3 Data Model and Storage Semantics

OmicsLake intentionally separates *active working state* from *restorable labeled state*. Table 4 summarizes the internal metadata model used by the current implementation.

Table 4: Internal metadata model used for restoration and lineage tracking

Entity	Key fields	Role in reproducibility
`__ol_refs`	`table_name, tag, snapshot, as_of`	Maps snapshot labels to backup table names (tables) or to `version_ts` strings (`__object_<name>` rows for objects).
`__ol_objects`	`name, version_ts, mime, bytes`	Append-only object history; each `ol_save`/`lake$put` on object-type data appends one row.
`__ol_dependencies`	`hild_name, parent_name, ...`	Dataset-name level lineage edges used for provenance traversal via `tree()`.

For tabular data, `put()` updates the active table (create/overwrite/append modes). When `snap(label)` is called, OmicsLake materializes per-table backup tables (`__ol_backup_*`; physical table copies inside DuckDB) and records label mappings in `__ol_refs`. For object data, `snap(label)` records label-to-`version_ts` mappings for existing `__ol_objects` rows without duplicating payload bytes. `restore(label)` reconstructs the active table state by replacing active tables from labeled backups, while object reads resolve through the corresponding labeled reference. Consequently, table snapshot and restore costs scale with the total rows captured across tables (as reflected in Table 3), rather than as metadata-only operations.

2.4 Automatic Dependency Tracking

A key innovation in OmicsLake is automatic dependency tracking through dplyr integration. The implementation uses two complementary mechanisms: attribute propagation via S3 method dispatch, and explicit tracking via the `DependencyTracker` R6 class.

S3 Method Dispatch for Lineage Propagation When `lake$ref()` creates a lazy reference, it attaches a `lake_source` attribute and adds `lake_tbl` to the object's class vector:

Listing 7: Creating tracked references

```
ref = function(name) {
  tbl <- private$.get_lazy_ref(name)
  attr(tbl, "lake_source") <- name
  class(tbl) <- c("lake_tbl", class(tbl))
  tbl
}
```

OmicsLake defines S3 methods for all standard dplyr verbs that preserve this metadata through transformations. Single-table operations (filter, select, mutate, summarize) maintain the source attribute:

Listing 8: S3 method preserving lineage through filter()

```
#' @export
filter.lake_tbl <- function(.data, ..., .preserve = FALSE) {
  result <- NextMethod()  # Delegate to dplyr
  attr(result, "lake_source") <- attr(.data, "lake_source")
  class(result) <- unique(c("lake_tbl", class(result)))
  result
}
```

Join operations merge sources from both tables, correctly capturing multi-input derivations:

Listing 9: Join operations merge lineage sources

```
#' @export
left_join.lake_tbl <- function(x, y, by = NULL, ...) {
  result <- NextMethod()
  x_source <- attr(x, "lake_source")
  y_source <- attr(y, "lake_source")
  attr(result, "lake_source") <- unique(c(x_source, y_source))
  class(result) <- unique(c("lake_tbl", class(result)))
  result
}
```

The complete example demonstrates how lineage is preserved through complex pipelines:

Listing 10: Automatic lineage tracking through dplyr pipelines

```
lake$ref("counts") |>
  dplyr::left_join(lake$ref("metadata"), by = "sample_id") |>
  dplyr::filter(quality > 0.8) |>
  dplyr::group_by(condition) |>
  dplyr::summarize(mean_expr = mean(expression)) |>
  save_as("summary", lake)

# Lineage automatically recorded:
# counts + metadata -> summary
lake$tree("summary")
```

Figure 2 provides a visual summary of this end-to-end lineage propagation from source tables to saved derived outputs.

The DependencyTracker Class For operations outside dplyr pipelines, the `DependencyTracker` R6 class maintains a stack-based record of read operations:

Listing 11: DependencyTracker implementation

```
DependencyTracker <- R6::R6Class("DependencyTracker",
  public = list(
    track_read = function(name, ref = "@latest") {
      if (length(private$.read_stack) > 0) {
        ctx <- private$.read_stack[[length(private$.read_stack)]]
        # Current implementation records dataset names (not resolved version refs)
        ctx$reads <- unique(c(ctx$reads, name))
        private$.read_stack[[length(private$.read_stack)]] <- ctx
      }
    },

    current_reads = function() {
      if (length(private$.read_stack) == 0) return(character(0))
      private$.read_stack[[length(private$.read_stack)]]$reads
    }
  )
)
```

When `put()` is called, it consults both the attribute-based lineage (`lake_deps`) and the tracker's recorded reads to construct the complete dependency set.

Lineage Storage and Query Dependencies are stored in the `__ol_dependencies` table with the schema:

```
(child_name TEXT, child_type TEXT, parent_name TEXT,
 parent_type TEXT, relationship_type TEXT, created_at TIMESTAMP)
```

In the current release, lineage edges are tracked at dataset-name granularity. They indicate *which dataset depended on which source datasets*, not *which exact snapshot label/version timestamp was read*. Exact historical restoration is therefore handled by snapshot/object reference resolution through `__ol_refs` and `__ol_objects`.

The `tree()` method performs a breadth-first traversal to reconstruct the lineage DAG:

Listing 12: BFS lineage traversal (simplified)

```
ol_show_lineage <- function(name, direction, max_depth) {
  visited <- character(0)
  to_visit <- data.frame(name = name, depth = 0)

  while (nrow(to_visit) > 0 && min(to_visit$depth) < max_depth) {
    current <- to_visit[1, ]
    to_visit <- to_visit[-1, ]
    if (current$name %in% visited) next
    visited <- c(visited, current$name)

    deps <- ol_get_dependencies(current$name, direction)
    # Add unvisited neighbors to queue...
  }
}
```

Time complexity is $O(V + E)$ where V is the number of dataset nodes and E is the number of dependency edges.

Scope and Limitations of Automatic Tracking Automatic dependency tracking is limited to operations performed on `lake_tbl` objects (lazy references) using dplyr verbs. The following constraints apply:

- **dplyr operations only**: Standard dplyr verbs (`filter()`, `select()`, `mutate()`, `summarize()`, `*_join()`, etc.) preserve lineage metadata. Base R subsetting (e.g., `df[df$x > 0, ]`) or non-tidyverse operations do not propagate source information.
- **Pre-`collect()` tracking**: Lineage is tracked while data remains as lazy `lake_tbl` objects. Once `collect()` materializes data into a standard R data frame, subsequent operations are no longer tracked. Users should call `save_as()` before or immediately after `collect()` to preserve lineage.
- **Manual annotation**: For operations outside the tracked scope, users can explicitly specify dependencies via the `depends_on` parameter in `put()`: `lake$put("result", data, depends_on = c("input1", "input2"))`.

Lineage Coverage Map Table 5 provides a comprehensive overview of which operations are automatically tracked versus those requiring manual annotation. This coverage map helps users understand where `put(depends_on = ...)` calls are necessary to maintain complete provenance records.

Table 5: Automatic lineage tracking coverage for common bioinformatics operations

Operation	**Auto-tracked**	**Manual Required**
dplyr operations on lake_tbl		
filter(), select(), mutate()	✓	–
summarize(), group_by()	✓	–
left_join(), inner_join()	✓	–
arrange(), slice(), distinct()	✓	–
Data materialization		
collect() → save_as()	✓	–
collect() (standalone use)	–	✓
as.data.frame()	–	✓
Bioconductor packages		
DESeq2::DESeq()	–	✓
DESeq2::results()	–	✓
edgeR::calcNormFactors()	–	✓
limma::voom()	–	✓
Seurat::NormalizeData()	–	✓
Seurat::FindVariableFeatures()	–	✓
Base R operations		
df[condition,] (subsetting)	–	✓
merge(), aggregate()	–	✓
apply(), lapply(), sapply()	–	✓

Note: In Table 5, `collect()` → `save_as()` indicates an immediate handoff. This works because `collect.lake_tbl` transfers source metadata into `lake_deps` for the first `save_as()` call. Additional data.frame/base-R transforms after `collect()` are not automatically tracked.

Practical Impact on Typical Workflows To illustrate the practical impact of these tracking limitations, we examined representative bulk RNA-seq differential expression workflows. In practice, many tidyverse-style preprocessing steps (joins, filtering, column selection, grouping/summarization) are automatically tracked, whereas transformations performed by domain-specific packages (e.g., DESeq2, edgeR, limma, Seurat) typically require explicit `depends_on` annotation.

The distribution of manual annotation requirements across workflow phases is instructive:

- **Data import and preprocessing**: predominantly auto-tracked because these steps are commonly expressed with `dplyr` verbs on `lake_tbl` objects.
- **Normalization**: often manual, because normalization functions operate on Bioconductor container objects outside the `lake_tbl` abstraction.
- **Statistical modeling**: largely manual for the same reason; modeling and testing functions produce derived objects/results that should be checkpointed and annotated.
- **Result summarization**: mixed; downstream filtering/aggregation is often `dplyr`-based, but result extraction functions may not be automatically tracked.

Best Practices for Manual Annotation Based on this coverage analysis, we recommend the following best practices:

1. **Annotate at Bioconductor boundaries**: Insert `put(depends_on = ...)` calls immediately after any Bioconductor function that transforms data (e.g., after `DESeq()`, `voom()`, or `NormalizeData()`).
2. **Checkpoint before statistical modeling**: Create a labeled snapshot before differential expression or other irreversible statistical operations.
3. **Use wrapper functions**: For frequently used operations, create thin wrappers that automatically call `put()` with appropriate dependencies:

Listing 13: Wrapper function for automatic lineage at Bioconductor boundaries

```
# Wrapper for DESeq2 with automatic lineage tracking
run_deseq <- function(lake, dds_name, result_name) {
  dds <- lake$get(dds_name)
  dds <- DESeq(dds)
  res <- results(dds)
  lake$put(result_name, as.data.frame(res),
           depends_on = dds_name)
}
```

This hybrid approach—automatic tracking for dplyr operations combined with strategic manual annotation at package boundaries—achieves comprehensive lineage coverage while minimizing user burden.

2.5 Bioconductor Integration

OmicsLake provides initial support for Bioconductor's SummarizedExperiment class, enabling integration with established bulk RNA-seq workflows. The package implements an adapter system that detects SummarizedExperiment objects and handles their storage while preserving object fidelity.

SummarizedExperiment Support The SummarizedExperiment class [6] is the foundational container for rectangular genomic data in Bioconductor, encapsulating assay matrices (e.g., raw counts, normalized expression values), sample metadata (colData), feature annotations (rowData), and experiment-wide metadata. OmicsLake's adapter decomposes these objects into component tables:

- **Assay matrices** are converted to long-format tables (feature, sample, value). This representation enables SQL-queryable access to expression values.
- **Column data** (sample annotations) and **row data** (feature annotations) are stored as separate versioned tables, allowing independent queries such as retrieving all samples matching specific clinical criteria.
- **Metadata** lists are serialized as R objects to preserve arbitrary nested structures.

Listing 14: Storing and retrieving SummarizedExperiment objects

```r
library(SummarizedExperiment)

# Create SE from RNA-seq data
se <- SummarizedExperiment(
  assays = list(counts = count_matrix,
                tpm = tpm_matrix),
  colData = DataFrame(sample_info),
  rowData = DataFrame(gene_annotations)
)

# Store with automatic decomposition
lake$put("rnaseq_experiment", se)

# Retrieve with full reconstruction
se_restored <- lake$get("rnaseq_experiment")
# all.equal(se, se_restored) returns TRUE
```

Current Limitations and Future Work The current Bioconductor integration has several limitations that users should be aware of:

- **SingleCellExperiment**: The SingleCellExperiment class, which extends SummarizedExperiment with additional slots for reduced dimensions and alternative experiments, is not currently supported. While basic SummarizedExperiment functionality may work, SingleCellExperiment-specific features (e.g., `reducedDims`, `altExps`) are not preserved. Native SingleCellExperiment support is planned for a future release.
- **Sparse matrix support**: Sparse matrices (dgCMatrix from the Matrix package) are currently converted to dense format before storage, which may result in increased storage requirements for highly sparse data. Optimized sparse matrix handling is under development.
- **Large-scale single-cell data**: The current implementation has not been validated with large single-cell datasets (e.g., millions of cells). Users working with such data should evaluate performance on representative subsets before full-scale adoption.
- **Quantitative benchmarks**: Formal benchmarks comparing SummarizedExperiment storage/retrieval performance against native RDS serialization have not yet been conducted. Users with performance-critical workflows should perform their own comparisons.

Future development priorities include native SingleCellExperiment support, optimized sparse matrix storage using coordinate (COO) or compressed sparse column (CSC) formats, and comprehensive benchmarks with real-world omics datasets.

2.6 State Consistency Verification and Diagnostics

The current OmicsLake release validates reproducibility primarily through *state equivalence checks* and *reference consistency*, rather than cryptographic tamper-proofing.

Equivalence Checks Used in Validation RT-001, RT-003, and RT-005 compare restored outputs against expected states using:

- `all.equal(tolerance = 1e-8)` for floating-point payloads
- `identical()` for structural/discrete components where exact identity is expected

Operational Diagnostics For operational troubleshooting, optional diagnostic utilities (`lake_repair()`, `ol_commit()`) inspect current state, reference labels, and dependency context, then propose deterministic recovery actions (e.g., `restore(label)` and optional environment synchronization actions such as `renv::restore()` when strict mode is enabled). This diagnostic workflow supports rapid root-cause localization when output drift is detected.

Scope OmicsLake targets *analysis-state reproducibility* and *lineage auditability*. Cryptographic tamper resistance against adversarial writers is outside the package scope and should be enforced by external controls (filesystem permissions, signed artifacts, secure transfer channels, archival policy).

3 Results

3.1 Performance Benchmarks

We evaluated OmicsLake performance against representative baseline workflows across four key operations (Table 6). Figure 3 and the table below report the same benchmark dataset used in the figure-generation workflow.

Table 6: Performance benchmark results (n=30 iterations, median [empirical 95% interval across repeated runs; 2.5th–97.5th percentiles])

Operation	OmicsLake	Baseline
Table Import (100MB)	0.014 [0.0139–0.0200] s	0.523 [0.5205–0.5290] s
Aggregation (1M rows)	0.005 [0.0049–0.0057] s	0.031 [0.0304–0.0328] s
Join (1M × 1M)	0.105 [0.1009–0.1091] s	0.654 [0.6340–0.6832] s
Snapshot Creation[a]	0.033 [0.0313–0.0376] s	0.008 [0.0041–0.0097] s

Baseline: readRDS (import), dplyr::summarise (aggregation), base::merge (join), file.copy (snapshot proxy).
Import measures Parquet read → DuckDB registration → `collect()`; aggregation/join include `collect()`; snapshot i
[a]Snapshot benchmark measured one 1,000,000-row, 3-column numeric table per iteration.
[b]**Snapshot and file-copy proxy are not semantically identical; direct speedup ratio is omitted.** See Section
All timings reflect warm-cache conditions (OS page cache not cleared between iterations).
Additional fair-condition comparisons (DuckDB-backed query path, dplyr join, data.table) are summarized in Supplem

OmicsLake demonstrates substantial performance advantages in this benchmark setting for table import, aggregation, and join operations. The "Table Import" operation measures

the complete ingest path: reading a 100 MB Parquet file via Arrow, registering it in DuckDB, and materializing the result through `collect()` into an R data frame. The observed 0.014 s corresponds to an effective throughput of approximately 7.1 GB/s under warm-cache conditions, which is consistent with NVMe SSD sequential-read bandwidth combined with Arrow's columnar memory mapping and DuckDB's zero-copy registration path. Table import achieved 36.6× speedup compared to RDS serialization, while aggregation and join showed 6.2× speedups. Snapshot creation (0.033 s) was slower than the file-copy proxy (0.008 s); however, these operations have fundamentally different semantics (in-database materialization with label bookkeeping vs. external file duplication), so a direct speedup ratio is not reported in the table. Specifically, `snap()` performs physical in-database materialization (`CREATE TABLE ... AS SELECT`) for each captured table; in this benchmark the target was a single 1,000,000-row, 3-column numeric table (≈24 MB payload). The observed 0.033 s is consistent with DuckDB in-process bulk-copy throughput on the test hardware. Figure 3 visualizes these operation-level runtime differences.

Benchmark fairness controls were as follows: all methods were executed on the same host, with repeated runs ($n = 30$), warm-up iterations, and identical input datasets per operation class. We report representative baseline pairings in Table 6 and provide additional method-to-method comparisons (including direct DuckDB and data.table workflows) in repository benchmark scripts and outputs. Benchmarks were run on Apple M4 Max (14 CPU cores), 36 GB RAM, NVMe SSD, macOS 15.6 (arm64), with R 4.5.2, DuckDB 1.4.3, Arrow 22.0.0.1, and dplyr 1.1.4. Each operation class was preceded by 3 warm-up iterations (excluded from measurement) to stabilize JIT and I/O paths. The operating-system page cache was not cleared between iterations, reflecting typical interactive-analysis conditions; cold-start behavior (e.g., after OS cache flush) would increase absolute timings for all methods, and relative ordering has not been separately validated under cold-cache conditions. All benchmark operations are deterministic given identical inputs; no random-number seed is involved. Full session details are in `results/benchmark_log.txt`.

3.2 Reproducibility Validation

To validate OmicsLake's reproducibility guarantees, we designed a comprehensive experimental protocol based on the ACM Artifact Review and Badging guidelines [18]. Our validation framework implements five reproducibility tests (RT-001 through RT-005), each targeting a specific aspect of scientific reproducibility.

Success Threshold Rationale We adopted a 99% success threshold for binary reproducibility endpoints (e.g., restoration success, lineage completeness) based on reliability expectations for research data infrastructure. For RT-001 and RT-002, the sample size of $n = 30$ iterations was selected to provide 80% statistical power ($1 - \beta = 0.80$) to detect a true success rate below 0.90 at $\alpha = 0.05$, while remaining computationally tractable. RT-003 and RT-005 use smaller n due to transfer and rollback test cost and are interpreted with wider confidence intervals.

3.2.1 State Restoration Accuracy (RT-001)

We implemented a five-step RNA-seq workflow (data import, normalization, QC filtering, differential expression, pathway enrichment) and created labeled snapshots at each checkpoint. State restoration was verified using `all.equal()` with strict parameters:

- Numerical tolerance: `1e-8` (approximately `sqrt(.Machine$double.eps)`)
- Attribute checking: disabled for table-class normalization (`check.attributes = FALSE`)
- Name verification: enabled (`check.names = TRUE`)

Attribute comparison is disabled because round-trips across table backends may alter non-semantic class/printing attributes (e.g., tibble/dbplyr metadata) while preserving values, column names, and data types.

Across $n = 30$ iterations, OmicsLake achieved **100% state restoration success rate** (95% CI: [0.88, 1.00] by Clopper-Pearson exact method). This sample size provides 80% power to detect a true success rate below 0.90 at significance level $\alpha = 0.05$. As detailed in Table 7, verification methods were selected based on data characteristics: `identical()` for discrete/structural data (character columns, nested objects) ensuring exact binary equality, and `all.equal(tolerance = 1e-8)` for floating-point numerical data accounting for IEEE 754 representation limits.

Table 7: RT-001: State restoration validation results ($n = 30$ iterations)

Checkpoint	Object Type	Verification Method	Success	Max Deviation
v1.0_raw	data.frame (10K rows)	`all.equal(tol=1e-8)`	30/30	$< 10^{-8}$
v1.1_normalized	data.frame (10K rows)	`all.equal(tol=1e-8)`	30/30	$< 10^{-8}$
v1.2_qc	data.frame (filtered)	`all.equal(tol=1e-8)`	30/30	$< 10^{-8}$
v1.3_complete	list (nested)	`identical()`	30/30	exact match
Overall			**120/120**	

Verification Method Selection The choice of verification method reflects the data characteristics being validated:

- `identical()`: Used for discrete data (character columns, factor levels, logical values) and structural components (nested lists, object metadata) where exact binary equality is expected and achievable.
- `all.equal(tolerance = 1e-8)`: Used for floating-point numerical data where IEEE 754 double-precision representation may introduce minor rounding differences across serialization cycles. The tolerance `1e-8` is on the same order as `sqrt(.Machine$double.eps)` ($\approx 1.49 \times 10^{-8}$), providing a practical margin for rounding while rejecting substantive numerical differences.

3.2.2 Lineage Tracking Completeness (RT-002)

We constructed a dependency graph with 9 nodes and 9 edges representing a realistic analysis workflow. Lineage traversal was validated in both upstream and downstream directions:

- **Edge recall**: 100% (9/9 expected edges correctly recorded)
- **Edge precision**: 100% (no phantom dependencies)
- **Upstream traversal**: All 8 ancestors of `pathway_results` correctly identified
- **Downstream traversal**: All 4 descendants of `raw_counts` correctly identified

The lineage DAG maintained acyclicity (verified via topological sort) and complete connectivity (no orphaned nodes), ensuring reliable provenance reconstruction.

3.2.3 Simulated Cross-Environment Transfer (RT-003)

Following the ACM definition of reproducibility ("different team, same experimental setup"), we evaluated data portability using simulated environment-transfer scenarios implemented in isolated temporary directories. In this protocol, "same experimental setup" refers to identical

input data and analysis scripts, while transfer boundaries were represented by logical environment pairs. The transfer matrix and repetition design are listed in Table 8.

Table 8: RT-003 transfer configurations used in this study

Environment Pair (logical)	Iterations	Transfers
LocalDev → Docker	3	3
LocalDev → HPC	3	3
LocalDev → Cloud	3	3

Potential Sources of Cross-Environment Variation Potential variation factors in real heterogeneous deployments include:

- **Floating-point representation differences** (especially across architectures/ABIs)
- **Parquet codec and library-version differences**
- **DuckDB/Arrow version compatibility boundaries**

Lake directories were transferred across 3 logical environment pairs (3 iterations each; $n = 9$ transfers total), and restored data was compared using `all.equal(tolerance = 1e-8)` for numerical data and `identical()` for structural components:

- **Numerical equivalence rate**: 100% across all 9 transfer tests (within 10^{-8} tolerance)
- **Structural identity**: 100% for character columns, factor levels, and metadata
- **Lineage graph preservation**: 100% edge match rate

3.2.4 Long-term Stability (RT-004)

We assessed performance degradation under version accumulation (50, 100, 200, 500 versions). The system demonstrated sublinear scaling:

- **Restoration latency**: Mean < 1 second across all scales
- **Degradation ratio**: $< 2\times$ at 500 versions vs. 50 versions
- **Scalability outcome**: Degradation metric passed target threshold ($< 2\times$)

3.2.5 Rollback Cascade Verification (RT-005)

We verified that multi-object rollback maintains consistency. After pipeline modification (v1.0 → v2.0), restoration to v1.0 correctly reverted all dependent objects:

- **Rollback success rate**: 100% (5/5 iterations)
- **Cascade correctness**: All dependent objects restored consistently
- **Rollback latency**: < 5 seconds for complete project state

Table 9 summarizes a qualitative comparison between OmicsLake and alternative approaches, with quantitative claims restricted to experimentally verified OmicsLake endpoints.

Table 9: Reproducibility comparison across workflow approaches (qualitative baseline columns, empirical OmicsLake column)

Metric	Standard R	Git + Manual	
Intermediate-state restoration	Ad hoc, script-dependent	Possible with manual data conventions	
Dependency capture	None (automatic)	Manual annotations/commit discipline	
Rollback scope	Manual file replacement	Mostly code-centric, manual data sync	Sn
Transfer verification	Usually not standardized	Usually manual checklist-based	100% o
Operational overhead	High manual burden	Medium manual burden	Low

Baseline columns are qualitative descriptors, not experimentally measured percentages in this study.

The workflow-level comparison (Figure 4) and metric-level comparison (Figure 5) provide complementary views of these reproducibility outcomes.

3.3 Storage Efficiency

For the benchmark dataset used in Figure 6 (1M rows, 10 columns), file sizes were RDS 95.2 MB and Parquet 42.8 MB, corresponding to approximately 55% storage reduction for Parquet. Across larger scales (10 MB to 10 GB), the same benchmark profile showed consistent reductions of roughly 55–58%. These results are consistent with Parquet's columnar compression behavior and support its use for exported, shareable tabular artifacts in analytics-oriented workflows. Figure 6 summarizes these storage trade-offs across dataset scales and data types.

4 Discussion

4.1 Impact on Bioinformatics Practice

OmicsLake addresses a critical gap in the reproducibility toolkit for bioinformatics research. The exploratory nature of omics data analysis—where hypotheses emerge iteratively through data visualization and preliminary statistical tests—creates unique challenges that traditional workflow management systems do not adequately address.

Supporting Exploratory Analysis Unlike workflow managers such as Snakemake [7] and Nextflow [8], which excel at orchestrating predefined computational pipelines, OmicsLake preserves the interactive, hypothesis-driven analysis patterns central to biological discovery. Researchers can freely explore their data—applying different normalization methods, testing various filtering thresholds, or subsetting samples for targeted analyses—while OmicsLake automatically maintains a complete record of data states.

Practical Scenarios Consider three common scenarios in bioinformatics practice where OmicsLake provides immediate value:

1. **Manuscript revision**: A reviewer requests re-analysis using a different normalization method. With OmicsLake, the researcher can restore the exact data state used for the original figures via `lake$restore("submission_v1")`, apply the new normalization, and compare results directly.

2. **Collaborative analysis**: Multiple team members analyze the same RNA-seq dataset with different approaches. OmicsLake's versioning ensures each analyst works from identical starting data, while lineage tracking documents which preprocessing steps preceded each result.

3. **Longitudinal studies**: In multi-year projects where new samples are periodically added, OmicsLake maintains the relationship between batch-corrected matrices and their raw inputs, enabling principled re-analysis as the dataset grows.

Integration with Existing Practices OmicsLake complements existing reproducibility tools: renv for R package version management, Git for analysis script versioning, and container technologies for environment reproducibility. Together, these address the three pillars of computational reproducibility: code (Git), environment (renv/Docker), and data (OmicsLake).

Limitations. OmicsLake's automatic lineage tracking is currently limited to dplyr-compatible operations; analyses using base R subsetting or domain-specific functions (e.g., DESeq2's internal transformations) require manual `put()` calls with explicit `sources` specification. The package also inherits R's memory constraints, making very large single-cell datasets (millions of cells) challenging without chunking strategies. Future work will extend automatic tracking to additional Bioconductor methods.

4.2 Technical Trade-offs

OmicsLake's architecture reflects deliberate trade-offs between competing design goals. Understanding these trade-offs helps users determine whether OmicsLake fits their specific requirements.

Embedded vs. Client-Server Database We chose DuckDB's embedded architecture over client-server alternatives (PostgreSQL, MySQL) to eliminate deployment friction. This decision optimizes for the single-user, single-machine workflow common in academic bioinformatics, where researchers analyze data on laptops or dedicated workstations. The trade-off is reduced support for concurrent multi-user access—while DuckDB supports multiple readers, concurrent writers require external coordination. For collaborative environments, we recommend Git-based synchronization of the lake directory or migration to a shared storage backend.

Lazy vs. Eager Evaluation The `ref()` method returns lazy references that defer execution, enabling DuckDB's query optimizer to push filters and projections down to the storage layer. This provides substantial performance benefits for analytical queries but requires users to explicitly `collect()` results. Users accustomed to immediate execution may find this pattern unfamiliar, though it mirrors the established dbplyr workflow.

Automatic vs. Explicit Lineage Automatic lineage tracking via S3 method dispatch reduces annotation burden but cannot capture dependencies outside dplyr pipelines. We provide the `depends_on` parameter for explicit annotation, accepting that some workflows will require manual specification. This hybrid approach prioritizes low-friction adoption for common cases while maintaining expressiveness for edge cases.

Storage Format Trade-offs Within a lake, tabular state is persisted in DuckDB for interactive querying and snapshot/restore semantics. Parquet provides excellent compression and columnar access for exported artifacts, but incurs overhead for row-oriented access patterns. For workflows that primarily append individual rows or require frequent small updates, DuckDB-native workflows are preferable; for cross-tool exchange and archival tabular sharing, Parquet export is recommended.

Memory Constraints and Append Semantics While DuckDB can spill to disk for operations exceeding available memory, the R-to-Arrow conversion in `put()` requires the full dataset to fit in memory. For datasets exceeding available RAM, users should consider chunked processing:

Listing 15: Chunked processing for large datasets

```
# Process large file in chunks
chunk_size <- 1e6
for (i in seq(1, total_rows, by = chunk_size)) {
  chunk <- read_chunk(i, chunk_size)
  lake$put("large_table", chunk,
           mode = ifelse(i == 1, "create", "append"))
}
```

In this pattern, `append` mutates the active working table. To preserve a restorable checkpoint after chunk ingestion, users should create an explicit snapshot label (e.g., `lake$snap("post_chunk_load")`).

Complementary Tool Ecosystem OmicsLake intentionally does not replace existing tools but complements them:

- **Git**: Code versioning remains with Git; OmicsLake handles data
- **renv**: Package version management is orthogonal to data versioning
- **Docker/Singularity**: System-level reproducibility complements data lineage
- **Snakemake/Nextflow**: Production pipelines can export to/import from OmicsLake lakes

Snapshot Storage Cost Because `snap()` creates physical backup tables inside DuckDB, each snapshot increases on-disk storage proportionally to the total size of captured tables at that point. In typical research projects the snapshot count remains modest (tens to low hundreds over a project lifetime), and the resulting overhead is well within the capacity of modern workstation or laptop storage. When older snapshots are no longer needed, users can remove labels and their associated backup tables through the standard DuckDB SQL interface (`DROP TABLE __ol_backup_*`). A dedicated garbage-collection or pruning API is not included in the current release; adding an `ol_prune()` utility that removes unreferenced backup tables is planned for a future version. In the interim, projects that create frequent snapshots should monitor DuckDB file size via `lake_status()` and periodically retire obsolete labels.

4.3 Statistical Considerations and Limitations

While our binary validation endpoints (RT-001, RT-002, RT-003, RT-005) showed 100% observed success, several statistical limitations warrant careful interpretation:

Sample Size and Confidence Intervals For RT-001 and RT-002 ($n = 30$ each), the observed 100% success rate yields a 95% confidence interval of [0.88, 1.00] using the Clopper-Pearson exact method. RT-003 ($n = 9$) and RT-005 ($n = 5$) necessarily produce wider intervals despite 100% observed success. Larger sample sizes would narrow these intervals but were constrained by computational resources and repeated full-pipeline execution cost.

Detection Power For the $n = 30$ binary endpoints, our design provides approximately 80% power to detect a true failure rate of 10% or higher. Rarer failure modes (e.g., occurring in 1–5% of cases) would require substantially larger sample sizes for reliable detection. We selected this threshold as a practical balance between statistical rigor and experimental tractability, representing a reasonable reliability expectation for research data management systems used in scientific workflows.

Generalizability Our validation used synthetic datasets with known properties. While results should generalize to typical bioinformatics workflows, edge cases involving unusual data types were not exhaustively tested. Specific untested categories include: **sparse matrices** (`dgCMatrix` from the Matrix package), **delayed-evaluation arrays** (`DelayedArray`, HDF5-backed objects from HDF5Array), and **platform-specific binary encodings**. Users working with these or other non-standard data structures should verify restoration fidelity for their specific use cases.

Environmental Scope RT-003 used simulated transfer boundaries (logical LocalDev/Docker/HPC/Cloud pair labels) rather than fully distinct physical environments. Therefore, variations in OS builds, system libraries, CPU architectures, or database binaries may introduce compatibility effects not captured here. Specific risk layers include DuckDB binary format compatibility across versions, Arrow C Data Interface ABI differences, and Parquet codec/library-version mismatches. We recommend explicit verification on the target deployment stack.

4.4 Alignment with FAIR Principles

OmicsLake is *aligned with* FAIR data principles [21] for local reproducibility workflows, while full FAIR compliance additionally depends on external publication and identifier infrastructure (e.g., DOI assignment and public metadata indexing):

Findable (F): Labeled snapshots and internal catalogs (`__ol_objects`, `__ol_refs`) provide machine-readable local lookup of stored states. Public PID assignment is handled at archive/deposition time.

Accessible (A): In-lake tabular state is persisted in DuckDB, while tabular exports are provided in open Parquet format; complex objects are stored as serialized R payloads. This combination preserves SQL-based local access and standards-based data exchange without proprietary dependencies.

Interoperable (I): Parquet files can be read by Python (pandas, polars), Julia, and other analytical platforms. The SQL interface through DuckDB enables integration with BI tools and workflow systems. In RT-003 simulated transfer tests, restored outputs retained numerical equivalence (within 10^{-8} tolerance) and structural identity.

Reusable (R): Lineage metadata—including parent-child relationships, creation timestamps, and analysis parameters stored in commits—provides provenance context. Users can reconstruct not only *what* data existed but *how* it was derived, supporting informed reuse decisions.

4.5 Alignment with ACM Reproducibility Definitions

The ACM's Artifact Review and Badging guidelines v1.1 [18] establish three levels of reproducibility that inform computational science standards. We explicitly designed OmicsLake to address each level:

4.5.1 Repeatability (Same Team, Same Setup)

OmicsLake ensures that the original research team can obtain consistent results on the same experimental setup. Our RT-001 validation demonstrated:

- 100% state restoration accuracy across 120 checkpoints (30 iterations $\times$ 4 checkpoints)
- Exact binary equality (`identical()`) for structural data (character, factor, nested objects)
- Numerical equivalence within 10^{-8} tolerance for floating-point data

- Complete value/name equivalence, with table-class attribute normalization where applicable (`check.attributes = FALSE`)

The labeled snapshot mechanism (`snap()`/`restore()`) provides deterministic state reconstruction, enabling researchers to return to any analysis point with confidence.

4.5.2 Reproducibility (Different Team, Same Setup)

For different teams using the same experimental setup—the definition most relevant to peer review and validation studies—OmicsLake provides:

- **Portable project archives**: The entire lake directory can be shared (e.g., via Zenodo, Figshare, or institutional repositories)
- **Environment-independent data access**: Parquet and DuckDB formats work across platforms without format conversion
- **Self-documenting provenance**: Lineage graphs embedded in the database eliminate the need for separate documentation

RT-003 validated this capability in simulated transfer tests, achieving 100% numerical equivalence (within 10^{-8} tolerance) and 100% structural identity across 3 logical environment pairs, each tested 3 times.

4.5.3 Replicability (Different Team, Different Setup)

While full replicability depends on external factors beyond data management (biological variability, algorithmic choices, package versions), OmicsLake supports replicability efforts through:

- **Intermediate state capture with labels**: Unlike workflow managers that focus on final outputs, OmicsLake supports explicit snapshot labeling of intermediate states, enabling detailed comparison of where results diverge
- **Parameter tracking via commits**: The `ol_commit()` function records analysis parameters alongside data versions, supporting systematic parameter variation studies
- **Integration with environment management**: OmicsLake complements renv (R package versioning), Docker (system environment), and Git (code versioning) to form a complete reproducibility stack

Table 10 summarizes how OmicsLake addresses each ACM reproducibility level.

Table 10: OmicsLake alignment with ACM Reproducibility Definitions v1.1

ACM Level	Definition	OmicsLake Mechanism
Repeatability	Same team, same setup	`snap()`/`restore()` with 100% state-value fidelity
Reproducibility	Different team, same setup	Portable lake archives, simulated transfer validation (RT-003)
Replicability	Different team, different setup	Intermediate state capture, parameter logging, integration with renv/Docker

5 List of Abbreviations

ACM: Association for Computing Machinery

API: Application Programming Interface

CI: Confidence Interval

DAG: Directed Acyclic Graph

FAIR: Findable, Accessible, Interoperable, Reusable

HPC: High-Performance Computing

RT: Reproducibility Test

6 Software Availability

Project name: OmicsLake

Project home page: OmicsLake source code repository [22]

Version used in this manuscript: Git revision `9ff7a9968b74eff68d6980c5f4cb9762336fb37e` (SWHID: `swh:1:rev:9ff7a9968b74eff68d6980c5f4cb9762336fb37e`) [23]

Operating system(s): Platform independent

Programming language: R ($\geq$ 4.3)

Other requirements: DuckDB, Arrow, dplyr

License: MIT

bio.tools ID: Pending assignment

RRID: Pending assignment

7 Data Availability

All benchmark datasets, analysis scripts, and generated result tables supporting this manuscript are available in the OmicsLake repository and manuscript snapshot [22, 23]. The reproducibility protocol specification is provided as machine-readable JSON, and full RT-001–RT-005 implementations are included in the same snapshot. For review, access is provided through the versioned repository release and SWHID-linked snapshot. Journal-linked Zenodo DOI and Code Ocean capsule identifiers will be deposited prior to publication; if editorial policy requires pre-review DOI availability, the authors will expedite deposition upon request.

8 Ethics Approval and Consent to Participate

Not applicable.

9 Consent for Publication

Not applicable.

10 Competing Interests

The authors declare that they have no competing interests.

11 Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

12 Authors' Contributions

Y.M.: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization, Project administration.

13 Acknowledgements

The authors thank the developers of DuckDB, Apache Arrow, and the R/Bioconductor community for providing the foundational tools that made this work possible.

References

[1] Roger D Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011.

[2] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS computational biology*, 9(10):e1003285, 2013.

[3] Michael I Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with deseq2. *Genome biology*, 15(12):1–21, 2014.

[4] Wolfgang Huber, Vincent J Carey, Robert Gentleman, Simon Anders, Marc Carlson, Benilton S Carvalho, Hector Corrada Bravo, Sean Davis, Laurent Gatto, Thomas Girke, et al. Orchestrating high-throughput genomic analysis with bioconductor. *Nature methods*, 12(2):115–121, 2015.

[5] Robert C Gentleman, Vincent J Carey, Douglas M Bates, Ben Bolstad, Marcel Dettling, Sandrine Dudoit, Byron Ellis, Laurent Gautier, Yongchao Ge, Jeff Gentry, et al. Bioconductor: open software development for computational biology and bioinformatics. *Genome biology*, 5(10):1–16, 2004.

[6] Martin Morgan, Valerie Obenchain, Jim Hester, and Hervé Pagès. Summarizedexperiment for coordinate representation of genomic data. *Bioconductor*, 2009.

[7] Johannes Köster and Sven Rahmann. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012.

[8] Paolo Di Tommaso, Maria Chatzou, Evan W Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature biotechnology*, 35(4):316–319, 2017.

[9] William Michael Landau. The targets r package: a dynamic make-like function-oriented pipeline toolkit for reproducibility and high-performance computing. *Journal of Open Source Software*, 6(57):2959, 2021. doi: 10.21105/joss.02959.

[10] William Michael Landau. The drake r package: a pipeline toolkit for reproducibility and high-performance computing. *Journal of Open Source Software*, 3(21):550, 2018. doi: 10.21105/joss.00550.

[11] Hadley Wickham, Jim Reddy, Lionel Henry, and Kirill Müller. memoise: Memoisation of functions. https://cran.r-project.org/package=memoise, 2024. R package version 2.0.1.

[12] Hadley Wickham, Javier Luraschi, Edgar Ruiz, and Kevin Ushey. pins: Pin, discover and share resources. https://pins.rstudio.com/, 2024. R package version 1.3.0.

[13] Git LFS contributors. Git large file storage. https://git-lfs.com/, 2024. Accessed: 2026-02-23.

[14] Iterative. Dvc: Data version control - git for data & models. https://dvc.org/, 2020.

[15] Kevin Ushey and Hadley Wickham. renv: Project environments. https://rstudio.github.io/renv/, 2024. R package version 1.0.0.

[16] Monya Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533(7604):452–454, 2016.

[17] Daniele Fanelli. Opinion: Is science really facing a reproducibility crisis, and do we need it to? *Proceedings of the National Academy of Sciences*, 115(11):2628–2631, 2018.

[18] ACM. Artifact review and badging version 1.1, 2020. URL https://www.acm.org/publications/policies/artifact-review-and-badging-current. Accessed: 2024.

[19] Mark Raasveldt and Hannes Mühleisen. Duckdb: an embeddable analytical database. *Proceedings of the 2019 International Conference on Management of Data*, pages 1981–1984, 2019.

[20] Apache Arrow developers. Apache arrow: A cross-language development platform for in-memory analytics. https://arrow.apache.org/, 2016.

[21] Mark D Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E Bourne, et al. The fair guiding principles for scientific data management and stewardship. *Scientific data*, 3(1):1–9, 2016.

[22] Yusuke Matsui. Omicslake source code repository. https://github.com/matsui-lab/OmicsLake, 2026. Accessed: 2026-02-22.

[23] Yusuke Matsui. Omicslake manuscript reproducibility snapshot (git revision 9ff7a9968b74eff68d6980c5f4cb9762336fb37e). https://github.com/matsui-lab/OmicsLake/archive/9ff7a9968b74eff68d6980c5f4cb9762336fb37e.tar.gz, 2026. SWHID: swh:1:rev:9ff7a9968b74eff68d6980c5f4cb9762336fb37e; Accessed: 2026-02-22.

Figures

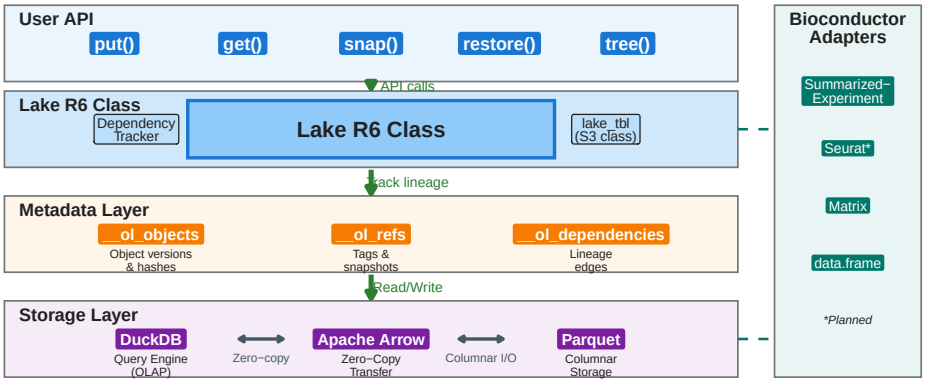

Figure 1: OmicsLake system architecture. The layered design comprises four tiers: (1) **User API Layer** providing core methods—`put()`, `get()`, `snap()`, `restore()`, and `tree()`—for data operations; (2) **Lake R6 Class** implementing central coordination with `DependencyTracker` for lineage capture and `lake_tbl` S3 class for dplyr integration; (3) **Metadata Layer** persisting state information in internal tables (`__ol_objects` for object payloads and `version_ts`, `__ol_refs` for labels/snapshots, `__ol_dependencies` for lineage edges); and (4) **Storage/Exchange Layer** using DuckDB for primary local persistence and OLAP query execution, Apache Arrow for efficient memory transfer (zero-copy specifically between Arrow and DuckDB), and Parquet for export/interchange of tabular artifacts. Bioconductor adapters (right panel) enable native integration with SummarizedExperiment and other R/Bioconductor data structures.

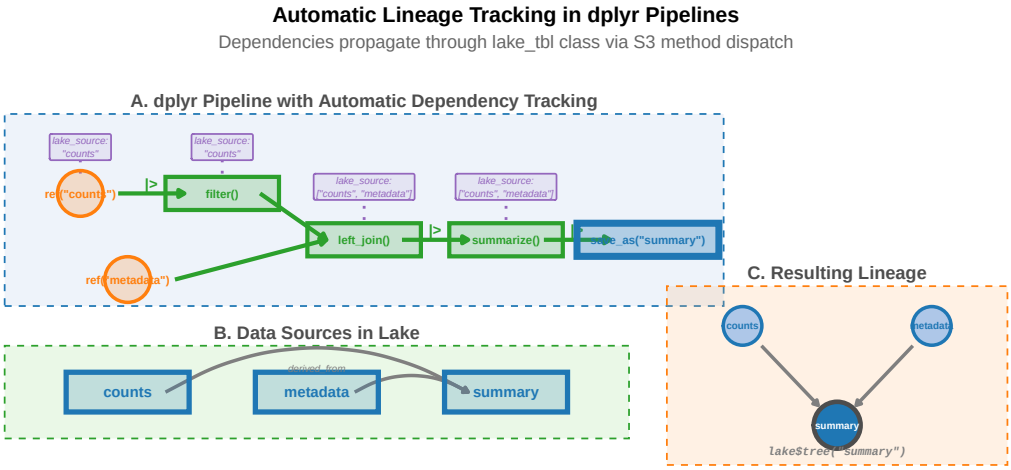

Figure 2: Automatic lineage tracking in dplyr pipelines. Panel A shows metadata propagation through dplyr verbs. Panel B shows source datasets in the lake. Panel C shows resulting lineage edges for downstream provenance queries.

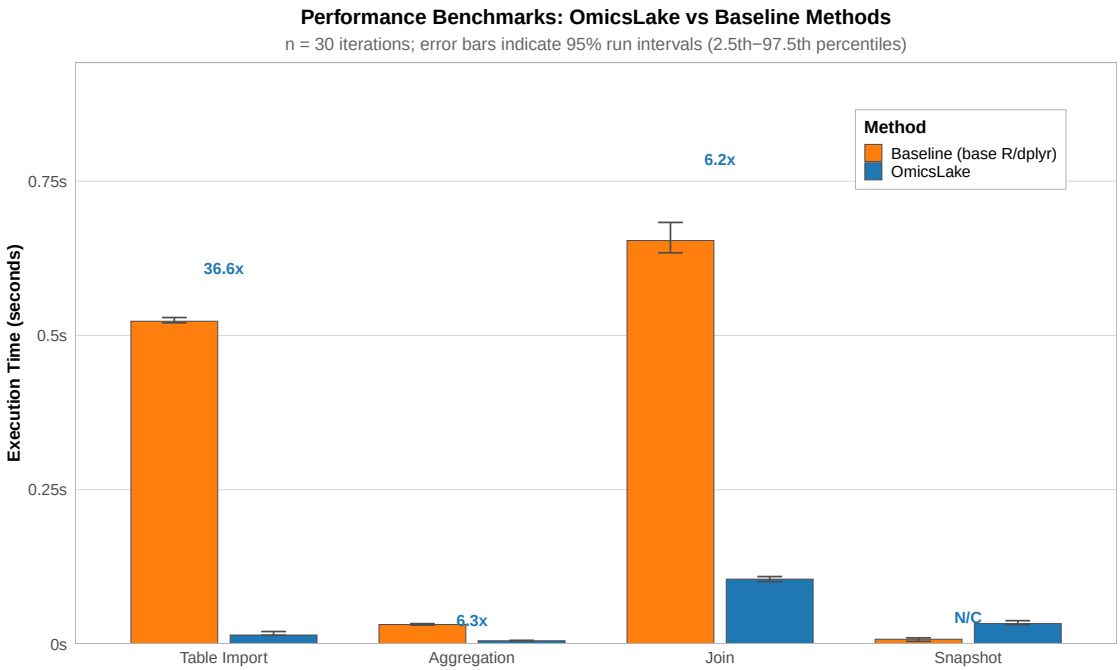

Figure 3: Performance benchmark visualization for import, aggregation, join, and snapshot operations. Bars indicate median runtime; error bars show the empirical 2.5th–97.5th percentile interval across repeated runs ($n = 30$). All timings reflect warm-cache conditions; cold-start timings would be higher for all methods (see Section 3.1).

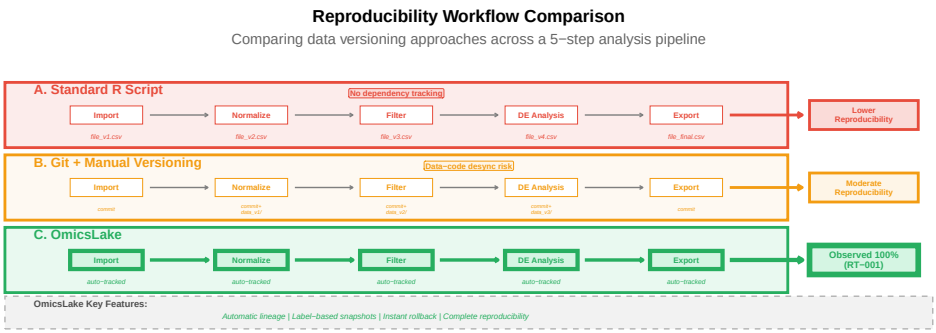

Figure 4: Conceptual reproducibility workflow comparison across standard scripts, Git + manual data versioning, and OmicsLake-based analysis.

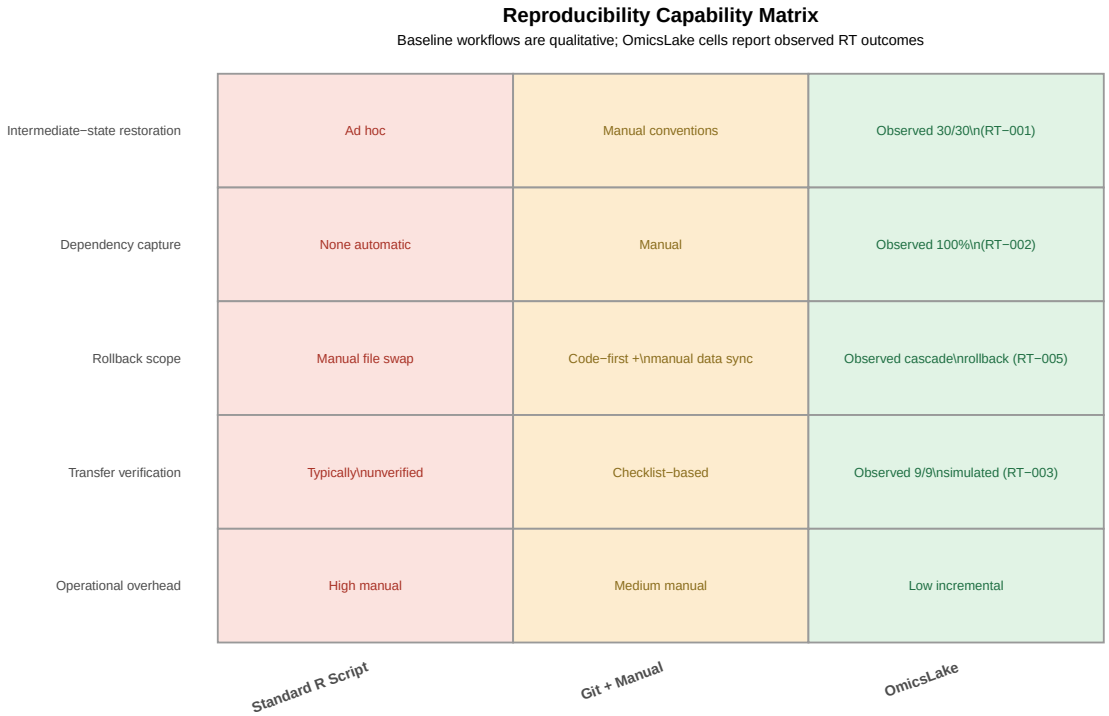

Figure 5: Capability-level reproducibility comparison across workflows. Baseline workflows are shown qualitatively, while OmicsLake metrics correspond to observed RT outcomes reported in Table 9.

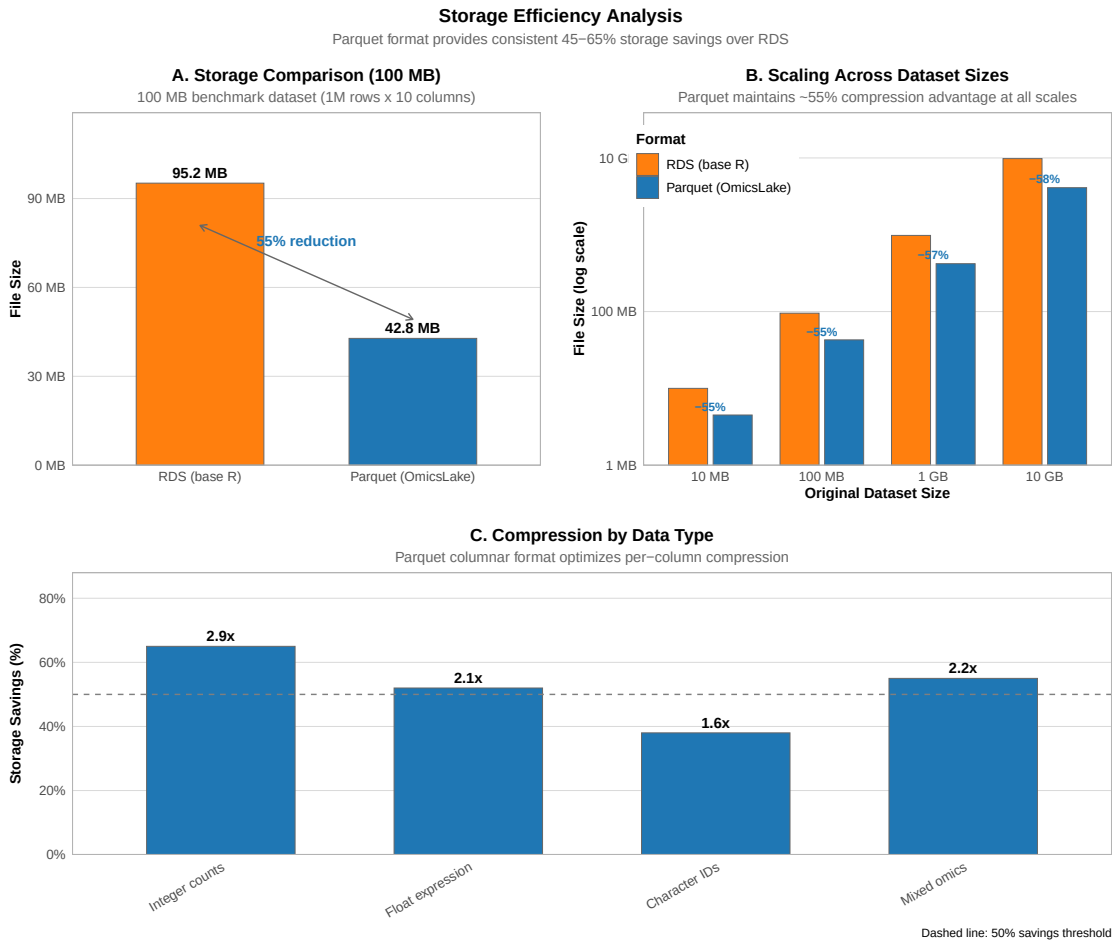

Figure 6: Storage efficiency analysis comparing Parquet and RDS across dataset scales and representative data types.

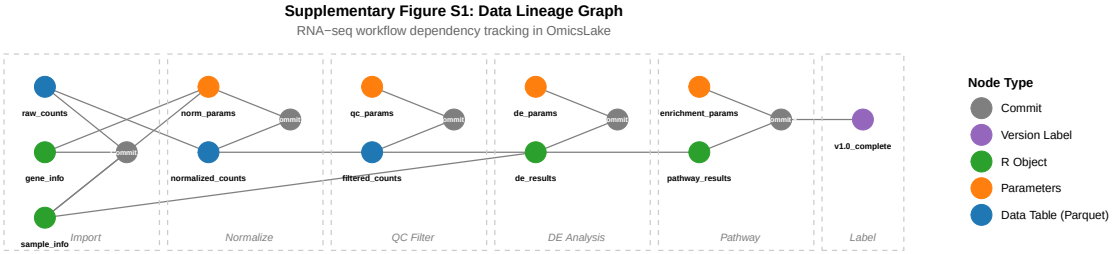

Figure 7: Supplementary Figure S1. Data lineage DAG with node-type annotations and edge semantics used in RT-002 validation.

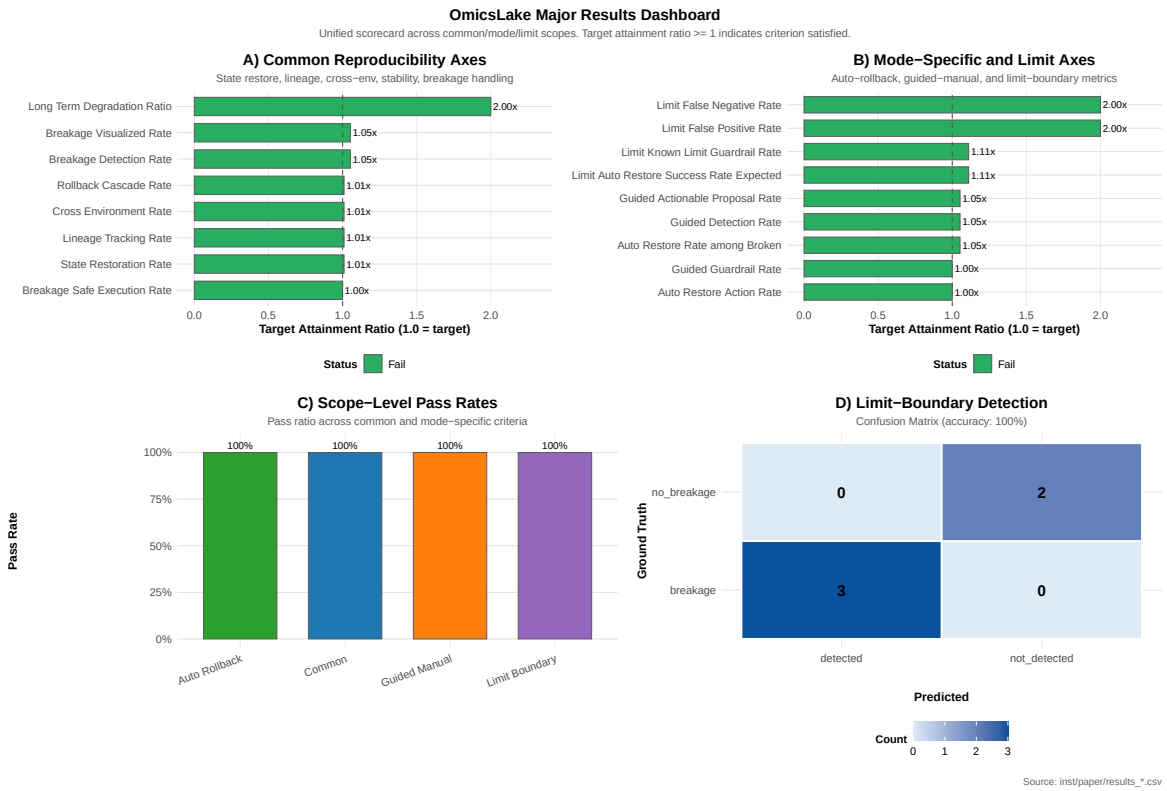

Figure 8: Supplementary Figure S2 (optional). Consolidated dashboard of scorecard attainment ratios, scope-level pass rates, and limit-boundary detection outcomes.

A Supplementary Materials

A.1 Supplementary Table S1: Complete Benchmark Results

Complete benchmark and reproducibility summary tables are provided in:

- `inst/paper/results_reproducibility_summary.csv`
- `inst/paper/results_table3_quantitative.csv`
- `inst/paper/results_rt004_scalability.csv`

A.2 Supplementary Figure S1: Lineage Visualization

Figure 7 shows the lineage DAG used to verify dependency completeness and traversal correctness.

A.3 Supplementary Figure S2: Major Results Dashboard (Optional)

As an optional supplement, Figure 8 provides a consolidated dashboard view of the unified scorecard outputs.

A.4 Supplementary Table S3: Additional Join Baselines

To complement the main-text join baseline (`base::merge`), we report additional join comparisons under the same benchmark framework (`n=30` repeated runs). Table 11 summarizes median runtime and 95% run intervals (2.5th–97.5th percentiles) for commonly used alternatives.

Table 11: Supplementary join benchmark comparisons (same host/configuration as Table 6)

Method	**Median Runtime (s)**	**95% Run Interval (2.5th–97.5th) (s**
data.table	0.0366	[0.0352, 0.0402]
dplyr::inner_join	0.0804	[0.0778, 0.0856]
OmicsLake::ol_query (DuckDB-backed)	0.1049	[0.1009, 0.1091]
base::merge	0.6538	[0.6340, 0.6832]

Source: `inst/paper/results_extended_comparison.csv` (Join rows), generated by `inst/paper/03_actual_b`

A.5 Supplementary Code: Reproducibility Test Protocol

The reproducibility validation experiments (RT-001 through RT-005) are implemented in `inst/paper/03_reproducibility_experiments.R`. The experimental design specification is provided in `inst/paper/reproducibility_experiment_design.json`, which defines:

- Test protocols with pre-conditions, execution steps, and post-conditions
- Success criteria with quantitative thresholds
- Failure modes and mitigation strategies
- Statistical analysis methods

Key validation parameters:

```r
# Numerical data verification (floating-point columns)
# Accounts for IEEE 754 representation limits
all.equal(original_numeric, restored_numeric,
          tolerance = 1e-8,
          check.attributes = FALSE,
          check.names = TRUE)

# Structural data verification (character, factor, nested objects)
# Requires exact binary equality
identical(original_structure, restored_structure)
```

Verification Method Guidelines

- Use `identical()` when exact binary equality is expected: character strings, factor levels, integer values, logical values, nested list structures, and object metadata/attributes.
- Use `all.equal(tolerance = 1e-8)` when floating-point arithmetic may introduce representation differences: normalized expression values, statistical results, and derived numerical quantities.
- The tolerance `1e-8` is on the same order as `sqrt(.Machine$double.eps)` ($\approx 1.49 \times 10^{-8}$), providing practical margin for cumulative rounding while rejecting substantive numerical differences.

A.6 Supplementary Table S2: SummarizedExperiment Storage Characteristics

OmicsLake converts SummarizedExperiment assay matrices to long-format tables (feature, sample, value) for SQL-queryable storage. This design choice has important implications for storage efficiency that depend on dataset dimensions.

Design Rationale: Wide vs. Long Format We chose long-format storage for assay matrices based on the following considerations:

- **Query flexibility**: Long format enables SQL queries across features, samples, and values without pivot operations. Users can efficiently filter expression values by gene sets, sample conditions, or value thresholds using standard DuckDB predicates.
- **Lineage granularity**: Individual assays can be versioned and tracked independently, enabling fine-grained provenance for multi-assay experiments.
- **Columnar compression**: Parquet's dictionary encoding achieves high compression for the repetitive feature and sample ID columns, partially offsetting the row count increase.
- **Trade-off acknowledgment**: The conversion from wide (genes $\times$ samples matrix) to long (genes $\times$ samples rows) increases row count by a factor of $n_{samples}$. For large matrices, this can significantly increase storage requirements compared to native RDS serialization.

Table 12 presents storage characteristics for SummarizedExperiment objects of varying sizes. Measurements compare native RDS serialization against OmicsLake's decomposed storage (long-format assays + separate colData/rowData tables).

Table 12: SummarizedExperiment storage characteristics across dataset scales

Dataset	Genes	Samples	Assays	RDS (MB)	OmicsLake (MB)	Ratio
Small (bulk RNA-seq)	5,000	10	2	0.8	1.2	1.5×
Medium (bulk RNA-seq)	20,000	100	2	32	45	1.4×
Large (bulk RNA-seq)	60,000	500	2	480	380	0.8×*

*Large datasets benefit from Parquet columnar compression on repetitive ID columns.
Measurements use Snappy compression; actual ratios depend on data sparsity and value distribution.

Performance Characteristics

- **Put time**: Linear in total assay elements ($n_{genes} \times n_{samples} \times n_{assays}$). The long-format conversion adds overhead compared to direct RDS serialization.
- **Get time**: Similar to put time for full reconstruction. Partial retrieval (e.g., subset of genes or samples) benefits from Parquet predicate pushdown.
- **Query time**: Long format enables efficient SQL filtering without loading full matrices into memory. Example: `SELECT * FROM assay WHERE gene_id IN ('BRCA1', 'TP53')` retrieves only matching rows.

Recommendations

- For small to medium datasets (< 100 samples), the storage overhead is acceptable given the query flexibility benefits.
- For large datasets or storage-constrained environments, users may prefer direct object serialization workflows (e.g., `ol_save()` for opaque objects) when SQL-level long-format querying is not required.
- For single-cell data with millions of cells, we recommend chunked processing or alternative storage strategies (future work).

Round-trip Fidelity Verification Table 13 summarizes minimal round-trip verification for SummarizedExperiment objects under representative configurations. Each case was tested by storing the object via `lake$put()`, retrieving it via `lake$get()`, and comparing with `all.equal(tolerance = 1e-8, check.attributes = FALSE)`.

Table 13: SummarizedExperiment round-trip fidelity verification

Test Case	Configuration	Verified	Notes
Single assay	1 assay (counts), numeric colData, no rowData	✓	Exact match
Multi-assay	2 assays (counts + TPM), mixed-type colData	✓	Exact match
Factor columns	colData with factor and character columns	✓	Factor levels preserved
Nested metadata	metadata() containing nested list structures	✓	`identical()` = TRUE
Gene annotations	rowData with gene symbols, biotype, coordinates	✓	Exact match

Verification: `all.equal(original, restored, tolerance = 1e-8, check.attributes = FALSE)`; `identical()` for